

Decisões do Pipeline Libras

Quatro frentes levantadas com dados do próprio repositório e testes na máquina: como reorganizar as categorias de regra, se vale montar a árvore sintática antes de buscar as regras, qual modelo roda de fato em uma A100, e o que seria preciso para treinar um modelo com conhecimento de língua de sinais.

215 regras extraídas (Libras) . 49 regras consolidadas (voice2sign) . 14 frases de Libras anotadas hoje . A100 80GB, driver 535 . rev. 18 ago 2026

1 As categorias misturam dois eixos diferentes

Dá para simplificar, sim – mas o problema principal não é a quantidade de categorias, e sim o fato de uma única coluna estar tentando responder duas perguntas incompatíveis.

O que os dados mostram

As 215 regras de Libras se distribuem em 11 categorias. A última coluna é o achado central: regras que dependem de **canal não-manual** (articulação-boca, expressão facial, marcação < >) aparecem em *todas* as categorias.

CATEGORIA	REGRAS	NAO-MANUAL	DIAGNOSTICO
morfologia	39	5 . 13%	absorve classificadores
sintaxe	38	11 . 29%	sobrepõe foco e pergunta
verbos	29	1 . 3%	coerente
expressoes-idiomaticas	21	5 . 24%	virou gaveta de sobras
pronomes	17	3 . 18%	coerente (IX)
tempo-aspecto	16	2 . 12%	parte é sintaxe, parte é verbo
foco	15	5 . 33%	mesmo fenômeno de pergunta
classificadores	15	4 . 27%	é morfologia
pergunta	13	6 . 46%	mesmo fenômeno de foco
negacao	9	6 . 67%	maioria é não-manual
concordancia	3	1 . 33%	fragmento de verbos
total	215	49 . 23%	

RAIZ

A categoria descreve o **domínio gramatical**, mas quem decide se a regra serve ao juiz de texto é o **canal**. Hoje isso está num booleano à parte (`verificavel_por_texto`) que já errou 20 vezes só no capítulo 5 da Gramática – todas regras de articulação-boca marcadas como verificáveis no texto.

Sobreposição concreta, não teórica

Dois pares reais de categorias diferentes descrevendo o mesmo fenômeno:

- **foco x pergunta** – “Construções de foco com duplicação final” (foco) vs. “Interrogativos em foco seguem estrutura de duplicação” (pergunta)
- **verbos x concordância** – “Verbos direcionais com concordância espacial” (verbos) vs. “Marcas não-manuais obrigatórias com verbos com concordância” (concordância)

Proposta: três eixos, cada um com uma pergunta só

EIXO	PERGUNTA QUE RESPONDE	VALORES	SERVE PARA
Canal	a regra aparece no texto da glosa?	manual / nao-manual / misto	separar voice2sign de sign2voice; substitui o booleano atual
Nível	que camada da língua?	sintaxe / morfologia / lexico	substitui as 11 categorias (colapsadas em 5)
Nó	onde na árvore a regra se aplica?	SUJ / PRED / V / COMPL / OBJDIR / OBJIND / PREP / NOME / ADJN / ADJV / S-INT[QU]	só para regras sintáticas; habilita o ponto 2

A consolidação do eixo *Nível* ficaria: **sintaxe** = sintaxe + foco + pergunta + posição de advérbio; **verbos** = verbos + concordância + aspecto verbal; **morfologia** = morfologia + classificadores; **pronomes** mantida; **lexico** = expressões idiomáticas, depois de tirar o que é não-manual e o que nem é regra (há entradas como “Técnicas de visualidade para treinamento de sinalização”, que é dica de ensino).

Usar só as produções da Tânia como categoria?

PARCIAL

Como esquema *único*, não – mas como eixo para o subconjunto sintático, é um encaixe quase perfeito e vale adotar.

As produções (SUJ, PRED, PREP, ADJN...) são **nós de uma árvore**. Isso funciona muito bem para as regras estruturais: “preposição locativa é nula” é literalmente uma regra do nó PREP; “adjetivo depois do nome” é NOME -> ADJN; “item-QU no fim” é ADJV.

Mas cerca de **metade das regras não vive num nó**: morfologia (39) e classificadores (15) descrevem o que acontece *dentro* de um sinal – ou seja, dentro de um terminal da árvore; e as 49 regras de canal não-manual acontecem *fora* da árvore. Forçá-las num nó criaria categorias falsas. Por isso os três eixos acima em vez de um só.

2

Árvore primeiro, regras depois

Das três frentes, esta é a de maior impacto – e temos evidência da rodada do juiz de que o desenho atual já está falhando exatamente onde ela resolve.

O problema, medido na rodada real

Hoje as 49 regras são despejadas inteiras no prompt e o modelo escolhe quais citar. Resultado da rodada com as 8 frases: LIBRAS.SINTAXE.1 (“ordem básica SVO”) foi citada em **7 das 8** frases – inclusive para julgar EU PODER IR PARA CASA HOJE, onde o que importava era a preposição, não a ordem. Sem âncora, o modelo cita a regra mais genérica que couber.

PROPOSTA

Etiquetar a glosa nos nós da produção *antes* de julgar, e então recuperar **apenas as regras dos nós presentes** naquela frase – em vez de oferecer a base inteira em todas as decisões.

```
// mesma frase, com a etapa de analise no meio
```

```
EU      PODER IR   PARA      CASA   HOJE  
SUJ     V         PREP      NOME   ADJN
```

```
nos presentes -- SUJ . V . PREP . NOME . ADJN
```

```
regras recuperadas (so as desses nos):
```

```
SUJ  -> sujeito pode ser nulo com referente presente
```

```
PREP -> preposicao locativa e sempre NULA na glosa    << dispara
```

```
ADJN -> adverbio de tempo pode modificar o locativo
```

```
veredito: invalido . regra citada: PREP->Nul . sugestao: EU PODER IR CASA HOJE
```

O que isso resolve

- **Recuperação dirigida em vez de despejo** – a regra citada passa a ser consequência do que está na frase, não da escolha livre do modelo.
- **Abstenção deixa de ser palavra e vira verificação** – nó presente na frase sem nenhuma regra associada é `nao_coberto` comprovável, não uma opinião do modelo.
- **Entrega a análise sintática pedida no início** – a etiquetagem por sinal é exatamente o “o que cada palavra é na glosa”, no padrão da Tânia.
- **Auditabilidade** – uma decisão errada fica visível na árvore, em vez de escondida no julgamento.

Riscos que não dá para varrer para debaixo do tapete

- **Propagação de erro** – árvore errada leva a regra errada. Passam a ser dois passos de LLM em vez de um, e os erros compõem. A contrapartida é que o erro fica *visível*: hoje ele já existe, só que invisível.
- **Cobertura estreita** – a gramática da Tânia tem só 6 sentenças transcritas. Frases fora desse conjunto vão cair em `nao_coberto` com mais frequência: honesto, mas reduz a taxa de decisão.

RECOMENDAÇÃO

Começar por **etiquetagem rasa** – qual nó cada sinal ocupa – e não pela árvore completa com todos os níveis de aninhamento. É bem mais robusta a erro e já entrega quase todo o ganho de recuperação. A árvore cheia, no formato do PDF da Tânia, vira um passo seguinte se fizer falta.

3

Modelo: a infraestrutura decide antes do benchmark

Antes de comparar qualidade, três restrições do nosso servidor eliminam a maior parte das opções que os rankings recomendam.

CORREÇÃO

O modelo em uso hoje nas pipelines **não é o Qwen3-8B** – é o Qwen3-30B-A3B-Instruct-2507. O 8B só aparece em pastas de experimento antigas (`experiments/qwen3_8b_v1`). A análise abaixo considera a substituição do 30B-A3B.

RETIFICACAO

A versão anterior deste documento concluiu que o driver 535 impedia rodar modelos de 2026 localmente. **Isso estava errado** e foi corrigido abaixo com teste executado na máquina: dá para rodar. O que quebra é CUDA **13.x**, não a versão do torch.

O que de fato restringe, medido na máquina

RESTRICAO	VALOR MEDIDO	CONSEQUENCIA REAL
Arquitetura da GPU	A100, compute 8.0 (Ampere)	sem FP8 e sem MXFP4 nativos; checkpoints nesses formatos ficam fora. VALE
Driver do cluster	535, CUDA 12.2	aceita qualquer wheel CUDA 12.x por compatibilidade de versão menor; só CUDA 13.x quebra, que é o caso do +cu130 citado no pyproject. NAO BLOQUEIA
Linha 4.x do transformers	encerrada na 4.57.6	não existe 4.58+; arquiteturas de 2026 exigem a linha 5.x, que pede apenas torch>=2.5. NAO BLOQUEIA

O teste que decide a questão

Montei um ambiente isolado com transformers 5.15.0 e torch 2.9.1+cu128 e rodei na A100 do cluster:

```
// 1. torch novo inicializa CUDA no driver 535?
torch: 2.9.1+cu128
cuda disponivel: True
device: NVIDIA A100-SXM4-80GB
matmul bf16 na GPU OK

// 2. transformers 5.15 reconhece a arquitetura?
qwen3_5 . qwen3_5_moe . qwen3_5_text . qwen3_5_vision      registradas

// 3. o checkpoint que ja esta no disco carrega?
AutoConfig.from_pretrained('Qwen/Qwen3.6-27B') -> Qwen3_5Config
  64 camadas . hidden 5120 . contexto 262.144 . bf16
tokenizer OK (vocab 248.077)
```

Conclusão: **a cadeia funciona**. Não precisa mexer no driver nem pedir nada ao administrador do cluster – o ajuste é nas dependências do projeto (transformers para 5.x e torch para um build cu128), e cabe num rebuild da imagem.

Candidatos que cabem em uma A100

MODELO	TIPO	PESOS BF16	SOBRA P/ KV	SITUACAO
Qwen3.8-27B ago/2026, Apache 2.0	denso 27,8B	55,6 GB	24,4 GB	recomendado ; precisa baixar (~11 min)
Qwen3.6-27B abr/2026, Apache 2.0	denso 27B	55,6 GB	24,4 GB	já no disco ; carregamento verificado
Gemma 4 31B	denso 31B	62,5 GB	17,5 GB	alternativa ; vence em multilíngue
Qwen3-30B-A3B-Instruct-2507 (atual)	MoE 30,5B, 3,3B ativos	~61 GB	~19 GB	funciona, mas é o mais fraco dos quatro
gpt-oss-120b	MoE 117B, MXFP4	~234 GB*	--	fora ; Ampere não tem MXFP4
Qwen3.5-122B-A10B	MoE 122B	~244 GB	--	fora ; não cabe nem em 2x A100

* sem suporte nativo a MXFP4, o carregamento desempacota os pesos para bf16 – é assim que 117B viram 234 GB.

2x A100

Segue sem valer a pena, agora por outro motivo: os melhores candidatos cabem com folga em **uma só** (24 GB livres para KV cache). O degrau seguinte (122B+) não cabe nem em duas. Mantém-se 1x A100 – e as outras sete livres para os colegas.

Qual é o ganho real de trocar

Não é “é mais novo”. O modelo atual ativa **3,3 bilhões de parâmetros por token** (é MoE); um denso de 27B ativa os 27 bilhões – cerca de **8x mais computação por token**, e é aí que aparece a diferença em aderência a schema e raciocínio sobre regra, que é exatamente o nosso uso.

Recomendação

10

Começar pelo Qwen3.6-27B, que já está no disco e cujo carregamento eu verifiquei – é o caminho com menos etapas até ter número na mesa. Subir transformers para 5.15 e torch para o build cu128 no ambiente do projeto.

20

Baixar o Qwen3.8-27B e comparar nos mesmos casos de teste. Mesmo tamanho, mesma folga de memória, quatro meses mais novo – a troca é barata e a comparação é honesta porque o resto do pipeline fica igual. Guardar o **Gemma 4 31B** como terceiro braço, já que os comparativos públicos o apontam melhor em multilíngue e nosso conteúdo é todo em português.

RESSALVA

O problema de qualidade que efetivamente observamos – a regra genérica de SVO citada em 7 de 8 frases – é de *arquitetura de recuperação* (ponto 2), não de capacidade do modelo. Trocar o modelo sem resolver o ponto 2 não corrige aquilo; conviria fazer as duas coisas, medindo uma por vez.

4

Treinar um modelo com conhecimento de língua de sinais

É viável – mas “treinar” quer dizer três coisas muito diferentes, e só duas delas fazem sentido aqui. O que decide não é a GPU, é o dado.

O que temos de dado hoje, contado

FONTE	VOLUME	NATUREZA	SERVE PARA TREINAR?
ASLG-PC12 (ASL)	81.091 pares	gerado por regra a partir de treebank	sim, mas é ASL e sintético
Pares ouro (ASL)	180 pares	revisado por humano	pouco para treino; bom para avaliar
Frases de teste (Libras)	8 frases	curado do PDF da consultoria	não
Árvores da consultoria (Libras)	6 sentenças	anotação manual especialista	não

Esse é o retrato honesto: para **ASL** temos volume; para **Libras**, que é o alvo, temos catorze frases somando tudo. Qualquer plano de treino de Libras começa resolvendo isso.

E existe dado de Libras lá fora – bastante

ACHADO

O **VLibrasBD** (pt-br2libras-gloss) tem **127.349 pares portugueses -> glosa Libras**, produzidos por uma equipe de dez intérpretes, em CSV UTF-8, licença CC BY-SA 4.0, DOI 10.17632/ryj88ckjww.2. Colunas: pt-br, libras-gloss, is_government_source, english_translation. É exatamente o par texto->glosa que o nosso Voice2Sign valida.

Há também o **Libras-UFPeI** (PROPOR 2026): 4.800 registros audiovisuais, dos quais 1.200 sentenças já lematizadas, anotadas em glosa e traduzidas – menor, mas com vídeo pareado, o que serve para o lado *sign2voice* que hoje deixamos de fora.

As três coisas que “treinar” pode significar

ABORDAGEM	DADO NECESSARIO	CABE EM 1x A100?	VEREDITO
Pré-treino continuado injetar Libras no modelo	bilhões de tokens no domínio	não; seriam meses de GPU	não viável e desnecessário: o VLibrasBD inteiro dá poucos milhões de tokens, duas ou três ordens de grandeza abaixo
Fine-tune LoRA/QLoRA na tarefa juiz, ou etiquetagem de nós	centenas a poucos milhares de exemplos rotulados da tarefa	sim, com folga; QLoRA em 27B ocupa bem menos que inferência bf16	viável ; gargalo é anotação, não GPU
Modelo dedicado texto->glosa tradução, não validação	127k pares já existem	sim	viável, outro objetivo : resolve gerar glosa, não julgá-la

Por que eu não faria fine-tune do juiz agora

Tem um motivo de arquitetura, não de custo. Você definiu no começo que a validação precisa **citar a regra e ser transparente**. Um modelo com as regras assadas nos pesos aprende a dizer “isto parece errado”, mas não consegue apontar *qual* regra foi violada – e perde a abstenção honesta, porque não há como distinguir “nenhuma regra cobre” de “não aprendi esse caso”. A base de regras explícita que montamos nas etapas 1 e 2 preserva essa trilha de auditoria; o treino a destruiria.

EXCEÇÃO

Onde o fine-tune encaixa bem é no **passo de etiquetagem do ponto 2** – dizer que nó cada sinal ocupa (SUJ, V, PREP...). É rotulagem de sequência, tarefa clássica, não precisa citar regra nenhuma, e umas 500 a 1.000 glosas anotadas já dariam um modelo específico bem melhor que prompt. Esse é o treino que eu faria primeiro.

O que eu faria com o VLibrasBD antes de pensar em treino

- **Substituir as 8 frases por um conjunto de avaliação de verdade** – com 127 mil pares dá para amostrar centenas de casos, inclusive por domínio (a parcela de conteúdo governamental é identificada por coluna).
- **Testar empiricamente as 49 regras extraídas** – “preposição locativa é sempre nula” virou regra a partir de dois livros; com 127 mil glosas reais isso deixa de ser afirmação e passa a ser medida. Regra que não se sustenta no corpus volta para discussão com a consultoria.
- **Gerar os casos inválidos a partir de dado real** – perturbar glosas reais (mover o item-QU, reinserir a preposição, inverter nome e adjetivo) produz negativos ancorados em português real, em vez de frases inventadas.

CUIDADOS

Duas ressalvas antes de tratar o VLibrasBD como gabarito: a licença é **CC BY-SA 4.0**, ou seja, derivado publicado herda a mesma licença – vale confirmar se isso é compatível com o destino do projeto; e a glosa dele segue as convenções dos intérpretes que o produziram, que **podem divergir das convenções da profa. Tânia**. Antes de usar como verdade, é preciso medir essa divergência numa amostra – senão a gente troca uma convenção por outra sem perceber.

Fontes

Dados de hardware, versões de biblioteca, tamanhos de modelo e contagens de dado foram medidos diretamente no servidor. Especificações de modelos: Hugging Face (Qwen3.8-27B, Qwen3.6-27B, documentação da família Qwen3.5), Google AI (releases do Gemma), discussão sobre MXFP4 exigir Hopper ou superior, Kaichup e Artificial Analysis (comparativos Qwen3.6 vs Gemma 4). Corpora de Libras: VLibrasBD / pt-br2libras-gloss em Mendeley Data (DOI 10.17632/rj88ckjww.2) e artigo em Data in Brief; Libras-UFPel Corpus (PROPOR 2026, ACL Anthology); V-Librasil (IEEE DataPort).